Day 6: Failed Authentication and Event Log Analysis

Generating and Locating Logon Failure Events in Windows Security Logs

Field	Details
Intern Name	Amrita Pathak
Date	08 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	11:59 PM IST

Contents

1. Objective 3
2. Tools and Environment 3
3. Key Terms 3
4. Why This Task Matters 4
5. Generating the Failed Authentication Attempts 4
6. Confirming the Account Was Not Locked 6
7. Opening Event Viewer and Navigating to the Security Log 6
8. Filtering for Event ID 4625 8
9. Reading a Failed Logon Event in Detail 10
10. Locating the Corresponding Successful Logon 13
11. Comparing the Failed and Successful Events 15
12. Preserving the Evidence 16
13. Findings 16
    13.1 The SubStatus code is more informative than the Status code alone 16
    13.2 Matching Logon Type across failed and successful events strengthens the benign explanation 16
    13.3 The first result in a filtered log is not automatically the relevant one 17
    13.4 Staying well under the lockout threshold left a wide safety margin 17
14. Conclusion 17

1. Objective

Generate several safe, intentional failed authentication attempts on my own lab account, without ever locking the account out, and then locate and examine the resulting events in the operating system's own logs. This exercise uses a Windows 11 lab endpoint and Windows Event Viewer, examining Security log events with ID 4625 (failed logon) and ID 4624 (successful logon) for direct comparison.

2. Tools and Environment

Item	Detail
Endpoint used	Windows 11 x64 lab VM
Account tested	CyberAmy (local account)
Log viewing tool	Windows Event Viewer (eventvwr)
Log examined	Windows Logs > Security
Key Event IDs	4625 (failed logon), 4624 (successful logon)
Evidence saved	4625 failed logins.evtx, 4624 login attempt.evtx

Windows was chosen for this task over the lab's Kali Linux machine. The reasoning was practical rather than arbitrary: Event ID 4625 is one of the single most recognized and frequently referenced security events in the entire industry, and generating a failed login on Windows requires no service setup at all (unlike Linux, where SSH would have needed to be started first), just locking the screen and mistyping a password. This made it the more direct, more broadly relevant choice for this specific exercise.

3. Key Terms

- **Authentication**: the process of proving identity to a system, typically by supplying a username and a matching secret (a password, in this case). A failed authentication event simply means this proof did not succeed on a given attempt.

- **Event ID**: a fixed numeric code Windows assigns to a specific, well-defined type of event. The same Event ID always means the same category of thing happened, regardless of which machine or which account is involved, which is what makes Event IDs searchable and comparable across an entire organization.

- **Logon Type**: a number recorded inside a logon event describing how the logon was attempted. The most common values are 2 (Interactive, typed directly at the machine's own keyboard and screen), 3 (Network, such as accessing a shared folder), 5 (Service, an automatic background process logging in, not a person), 7 (Unlock, entering a password to unlock an already-signed-in session), and 10 (RemoteInteractive, a Remote Desktop connection).

- **Status and SubStatus codes**: hexadecimal codes attached to a failed logon event. Status gives the broad category of failure; SubStatus gives the specific technical reason, and is usually the more informative of the two for an analyst.

- **Account lockout policy**: a security setting that automatically disables an account after a configured number of consecutive failed attempts, for a configured amount of time. Recent Windows 11 installations apply a default of 10 failed attempts within a 10-minute window on clean installs, which is the specific number this exercise was deliberately kept well under.

- **.evtx**: the native file format Windows uses to store exported event log data. Saving a filtered set of events as a .evtx file preserves the actual log data itself, not just a picture of it, meaning it can be reopened, re-filtered, or handed to another analyst later exactly as it was captured.

4. Why This Task Matters

Failed login attempts are one of the most common alert categories any SOC analyst will ever review, and also one of the easiest to misjudge without the right context. A single failed login rarely means anything on its own, people mistype passwords constantly. What actually matters is being able to read the specific detail inside the event:

- which account was targeted,
- what kind of logon was attempted, and
- precisely why it failed,

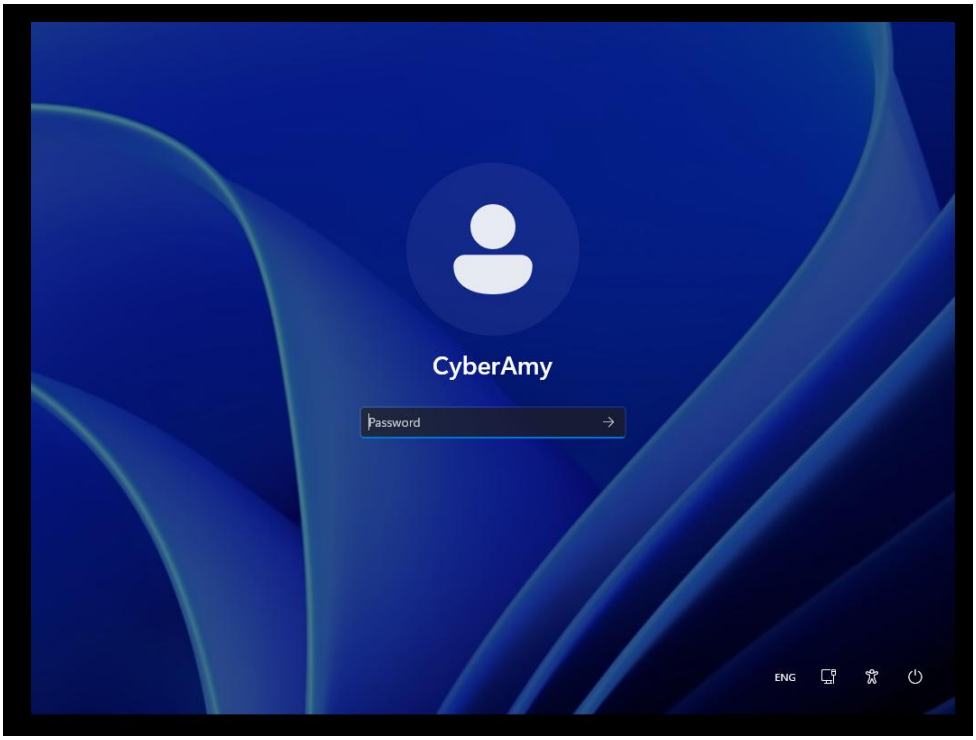
And then, they can judge whether that detail fits a pattern of legitimate human error or something more deliberate, like an attacker guessing passwords. This is exactly why the task asks for the events to be located and examined, not just generated. Anyone can mistype a password. The actual skill being built here is knowing where Windows records that event, how to filter a security log that can easily contain thousands of unrelated entries down to the handful that matter, and how to read the technical fields inside a single event well enough to state, with evidence, what actually happened.

The requirement to do this without locking the account out is also a deliberate part of the exercise, not just a safety note. A real investigation often needs to reproduce or test authentication behavior on a live system without disrupting it. Understanding exactly where an account lockout threshold sits, and staying comfortably under it, is itself a basic operational skill: the difference between safely testing something and accidentally causing an outage.

Windows specifically was the right target for this version of the exercise because Event ID 4625 carries outsized real-world relevance. Brute-force and password-spraying attacks against Windows accounts are common enough that 4625 is frequently one of the first Event IDs any new SOC analyst is taught to recognize by number alone, and the skill of reading a 4625 event's SubStatus code to distinguish a wrong password from a nonexistent username, seen directly in this exercise, is a real, transferable diagnostic technique used in actual investigations.

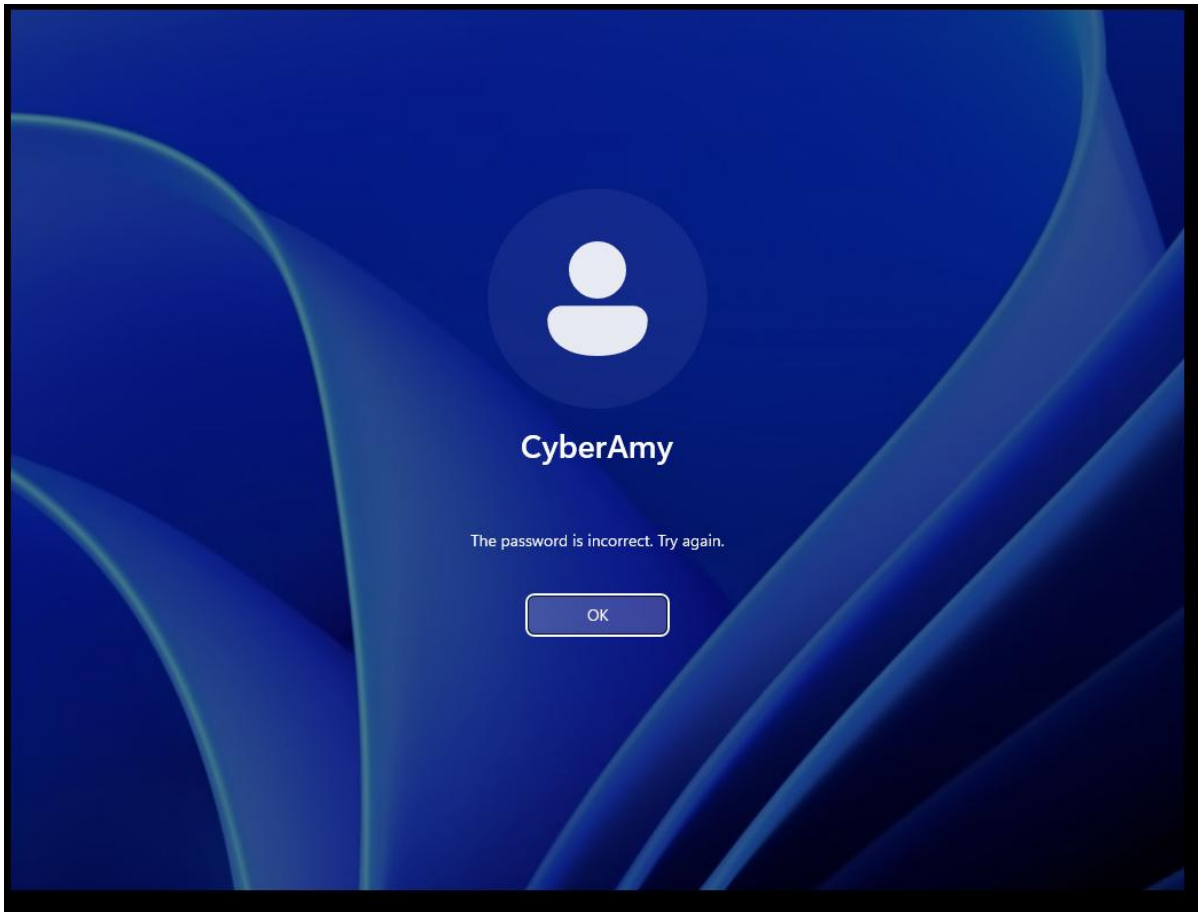
5. Generating the Failed Authentication Attempts

The Windows 11 VM was powered on from a shut-down state, which brought up the standard Windows sign-in prompt for the account CyberAmy directly, rather than an active session being locked. This is a genuine authentication prompt, not a simulation, entering a password here is processed by Windows exactly the same way any real sign-in is.



The sign-in prompt for account CyberAmy. Every character typed and submitted here, right or wrong, generates a real logon event in the Security log.

A deliberately wrong password was entered and submitted. Windows rejected it immediately with an on-screen error, and this exact process was repeated a total of four times, three fully intentional attempts plus one additional accidental mistype while trying to recall the correct password, which was left in as an honest part of the record rather than excluded, since it is a genuine and common real-world reason a 4625 event gets generated.

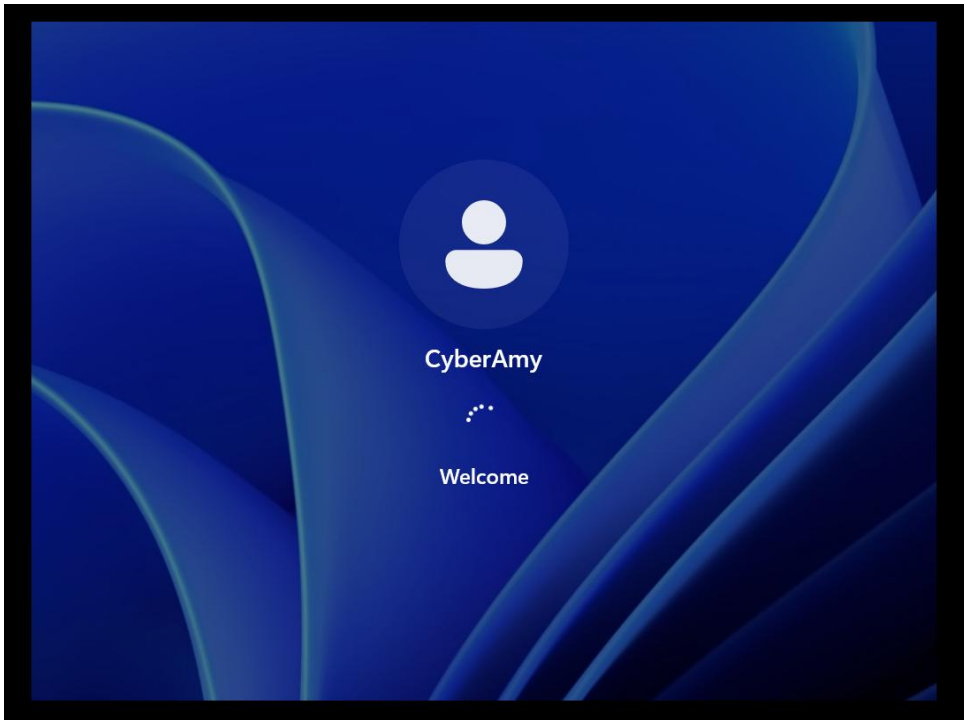


"The password is incorrect. Try again." This exact message, and the event it generates behind the scenes, repeated across all four failed attempts.

Each of these four attempts was kept well under the 10-attempt default lockout threshold discussed in the Key Terms section, at no point was there any real risk to the account.

6. Confirming the Account Was Not Locked

After the four failed attempts, the correct password was entered and submitted.

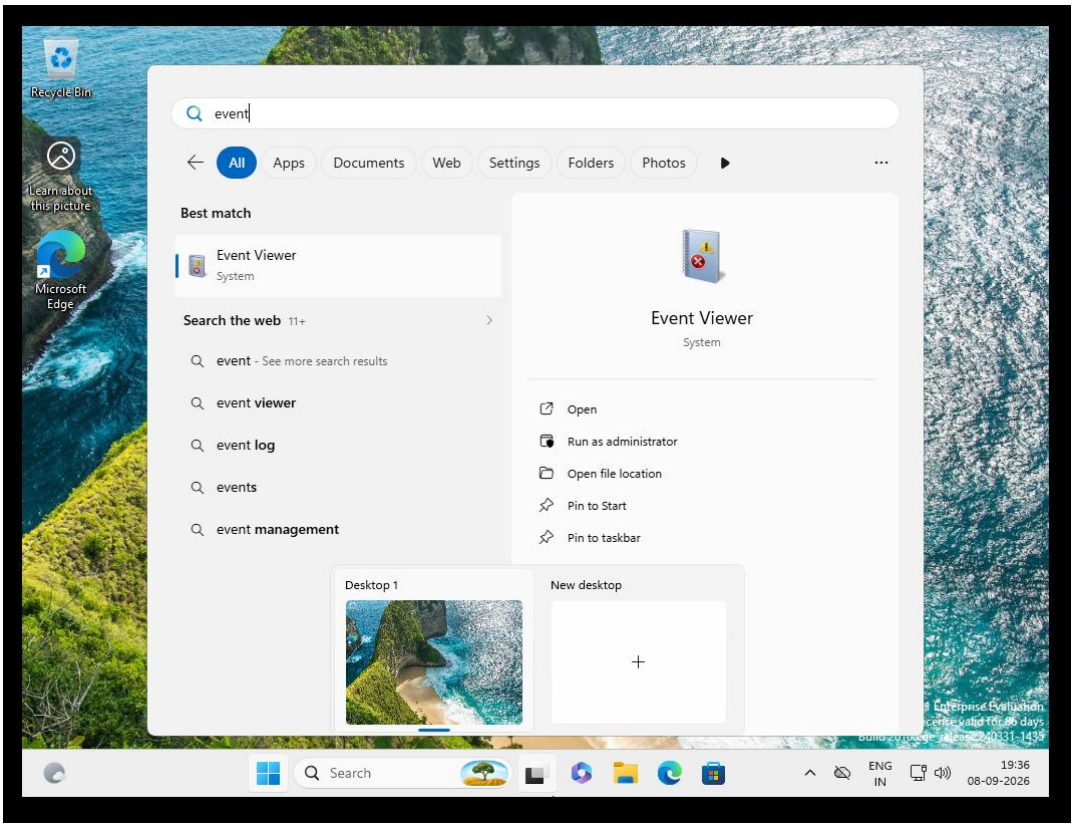


The "Welcome" transition screen immediately following the correct password, confirming the account signed in normally with no lockout, no warning, and no restriction of any kind.

This step matters as much as the failed attempts themselves for satisfying the task's requirement. Generating failed logins alone would not prove the account stayed usable, only a successful login immediately afterward proves that. This also created a second, distinct type of event (a successful logon, Event ID 4624) that would later be directly compared against the failed ones.

7. Opening Event Viewer and Navigating to the Security Log

Event Viewer is the built-in Windows tool for reading every log the operating system keeps, no separate installation is required. It was opened by clicking the Start button, typing "Event Viewer," and selecting it from the search results, then choosing "Run as administrator" specifically to guarantee full read and export permissions on the Security log from the outset.

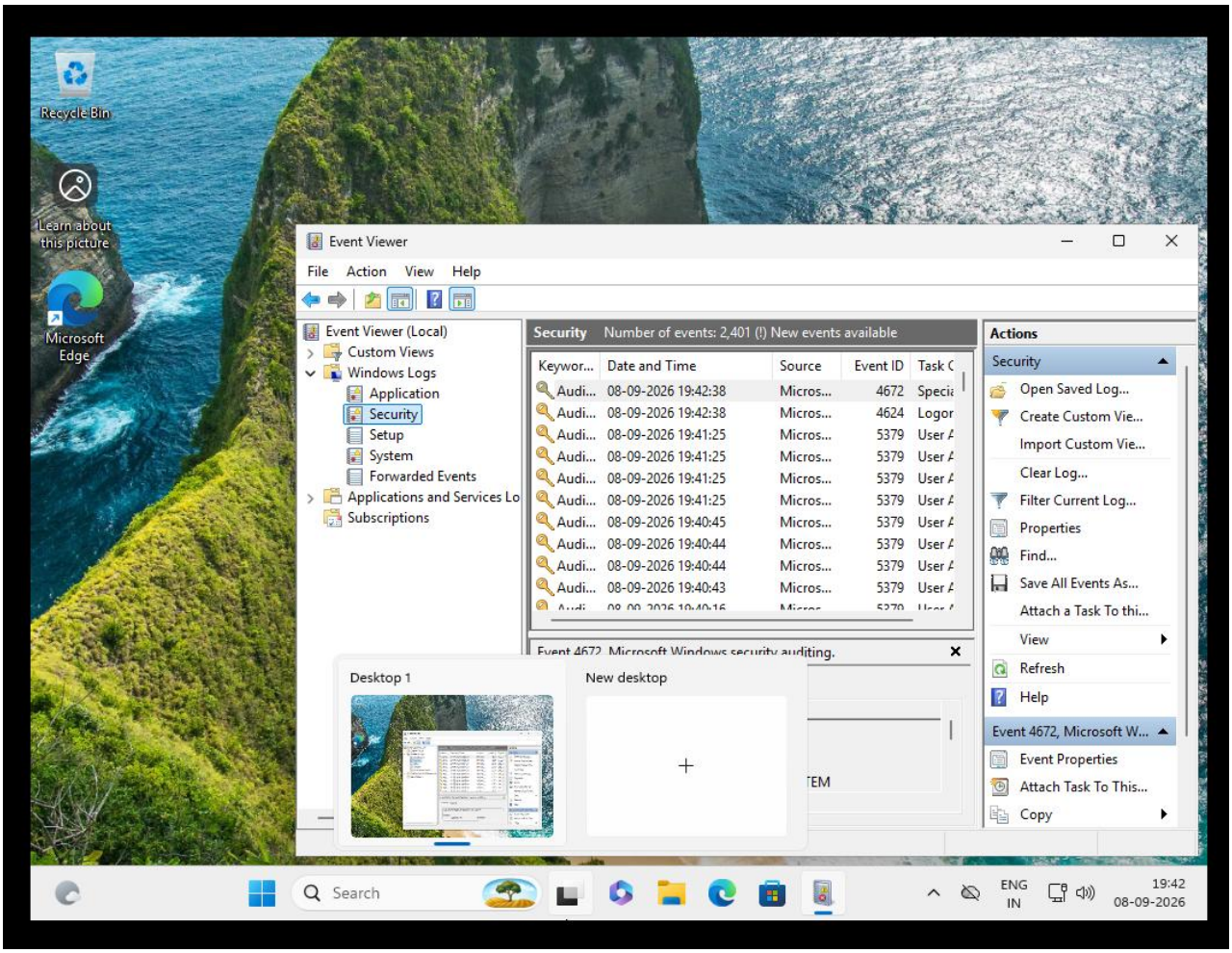


Locating Event Viewer through the Start menu search. Windows recognized it immediately as the best match.

Windows keeps several separate logs rather than one combined log:

- Application (events from installed software),
- System (events from Windows itself, drivers, hardware),
- Setup (installation-related events), and
- Security (every logon, logoff, and privilege-related event, including the ones this task needed)

Expanding "Windows Logs" in the left panel and clicking "Security" loaded the correct log.

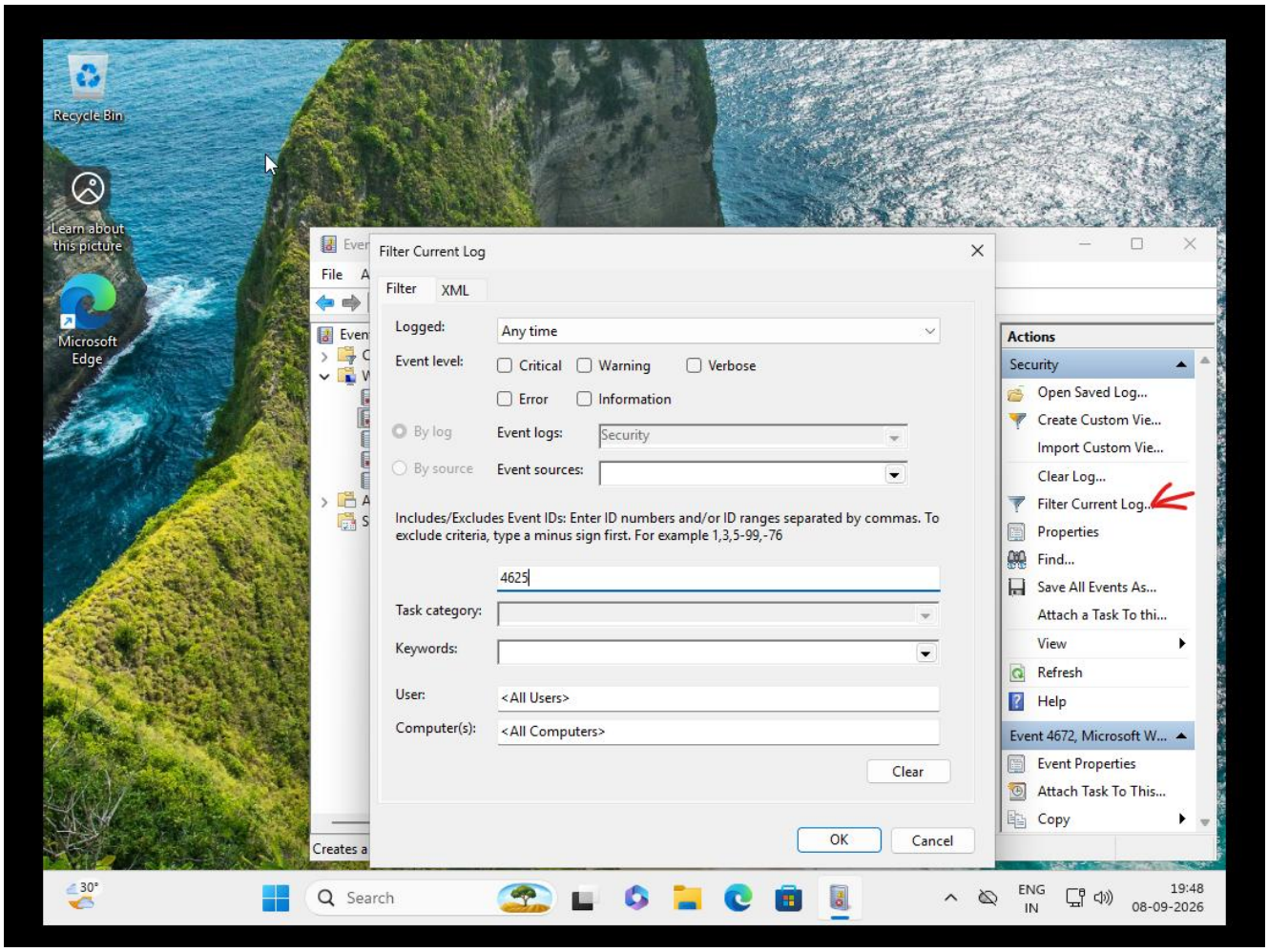


The unfiltered Security log: 2,401 total events. The very first row already shows an Event ID 4624 at 19:42:38, though at this stage it was not yet clear whether that specific entry was the relevant one.

With over 2,400 events in this log, most of them routine background activity generated automatically by Windows itself, scrolling manually to find four specific failed logins would have been impractical. This is exactly the situation Event Viewer's filtering feature is built for.

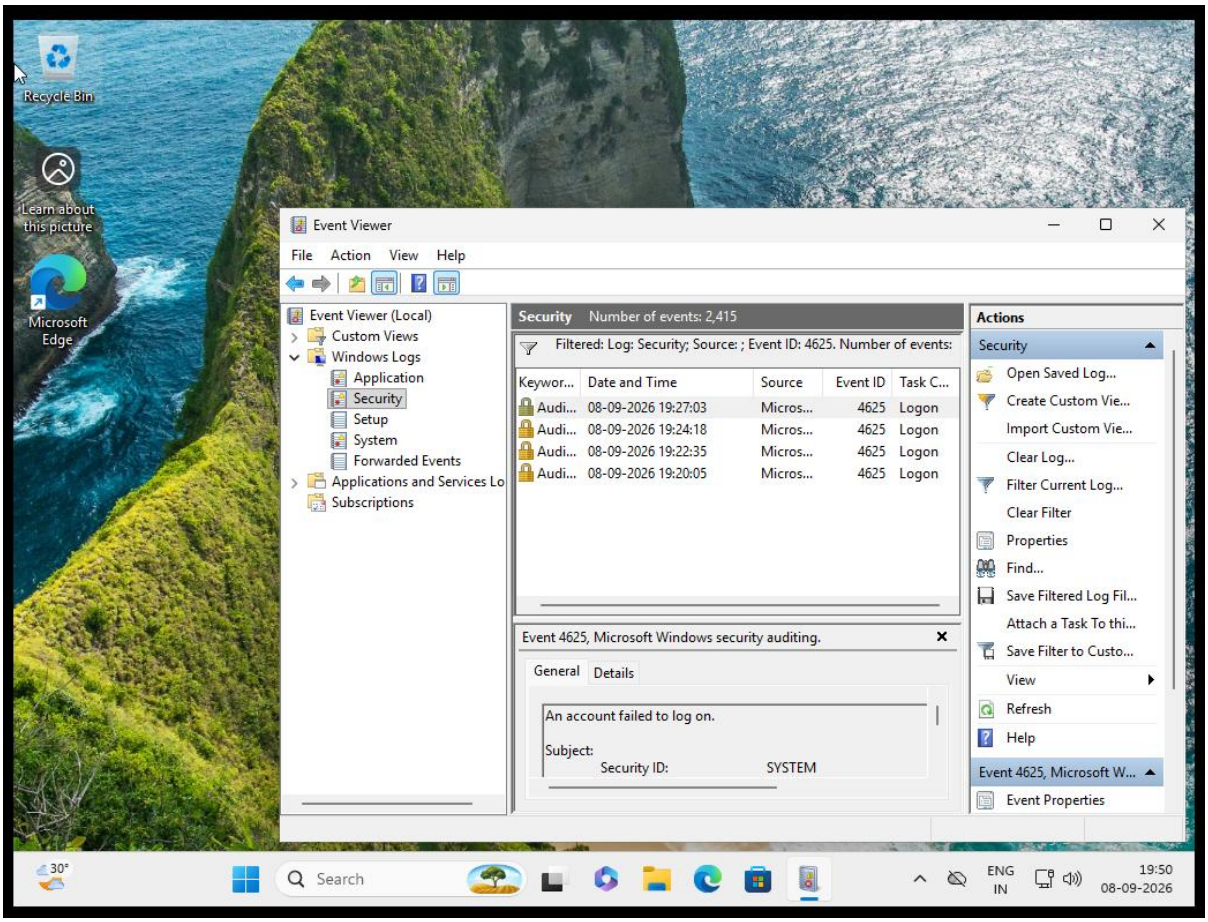
8. Filtering for Event ID 4625

From the Actions panel, "Filter Current Log..." was selected, and 4625 was entered into the Event ID field.



The Filter Current Log dialog with 4625 entered. This single filter is what narrows 2,401 unrelated events down to only failed logon events.

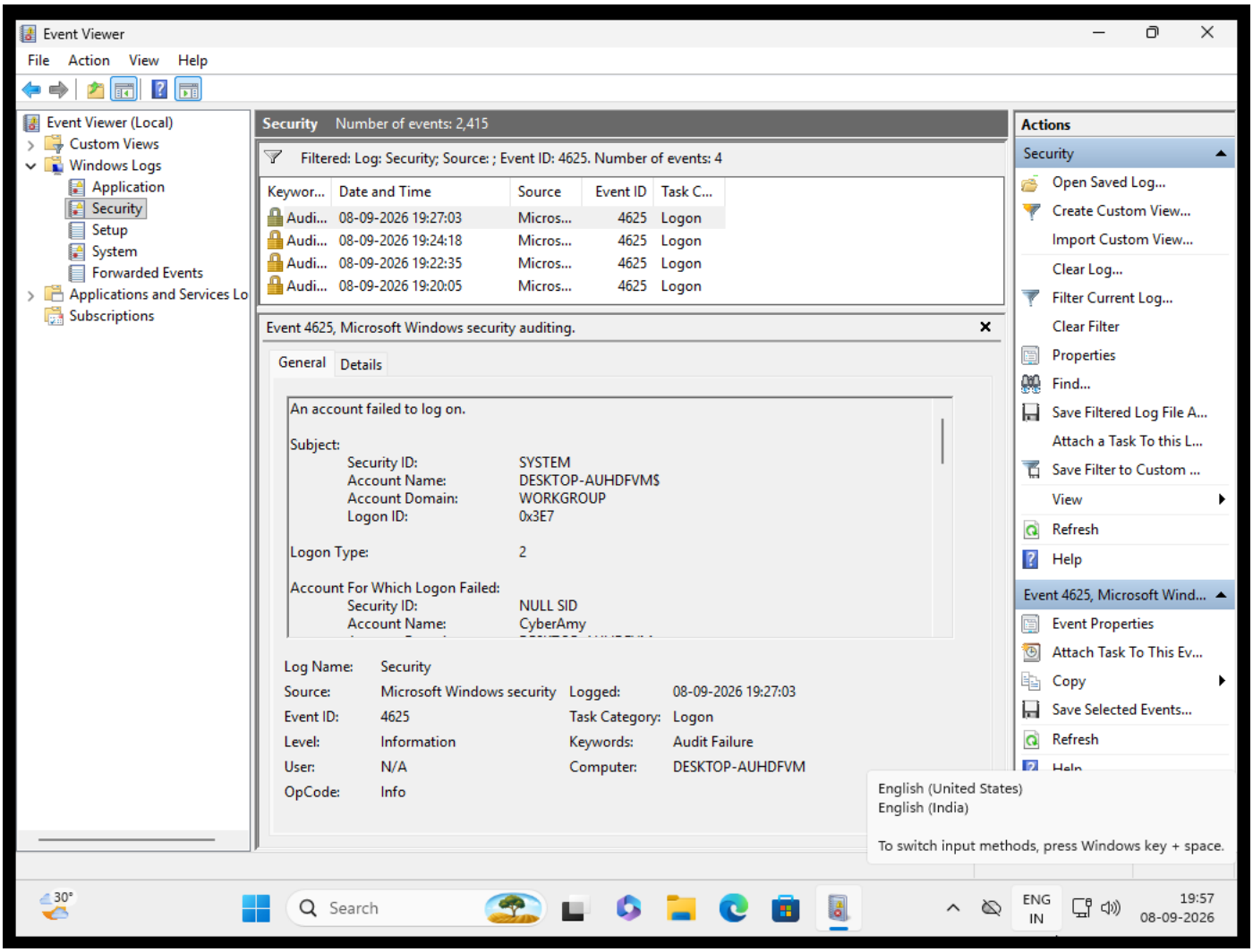
Filtering by Event ID works because every event of a given type is tagged with that number at the moment Windows writes it to the log, so the filter does not need to search event text or guess at meaning, it matches an exact, structured field. This is far more reliable than trying to search for a word like "failed" in the log's text, which could miss events or match unrelated ones.



Filtered result: exactly 4 events, Event ID 4625, spanning from 19:20:05 to 19:27:03, a roughly 7-minute window. This count matches the real number of failed attempts made, confirming nothing was missed and nothing extra was picked up.

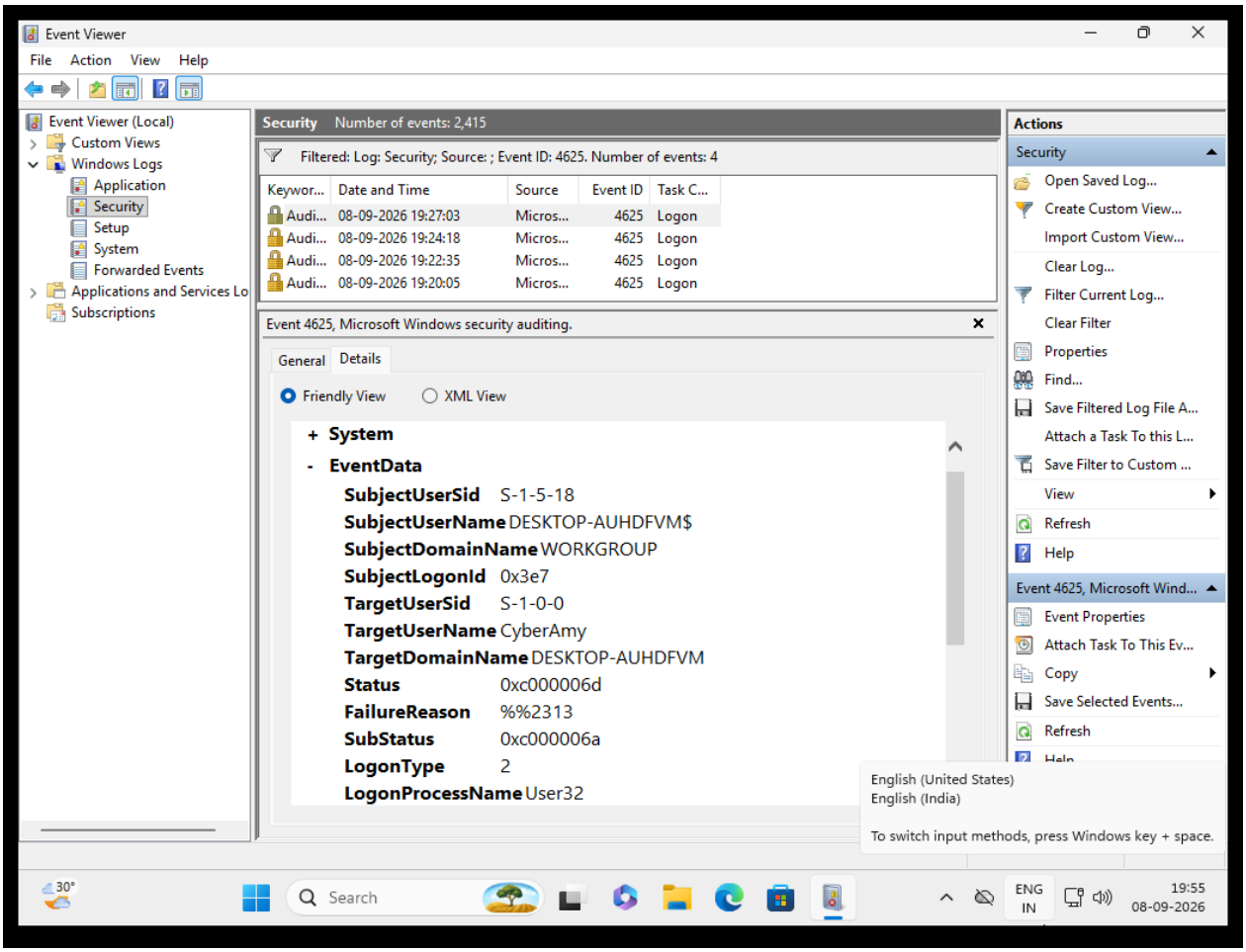
9. Reading a Failed Logon Event in Detail

Clicking on one of the four filtered events and reading its General tab gives the full plain-language description Windows generates for that event.



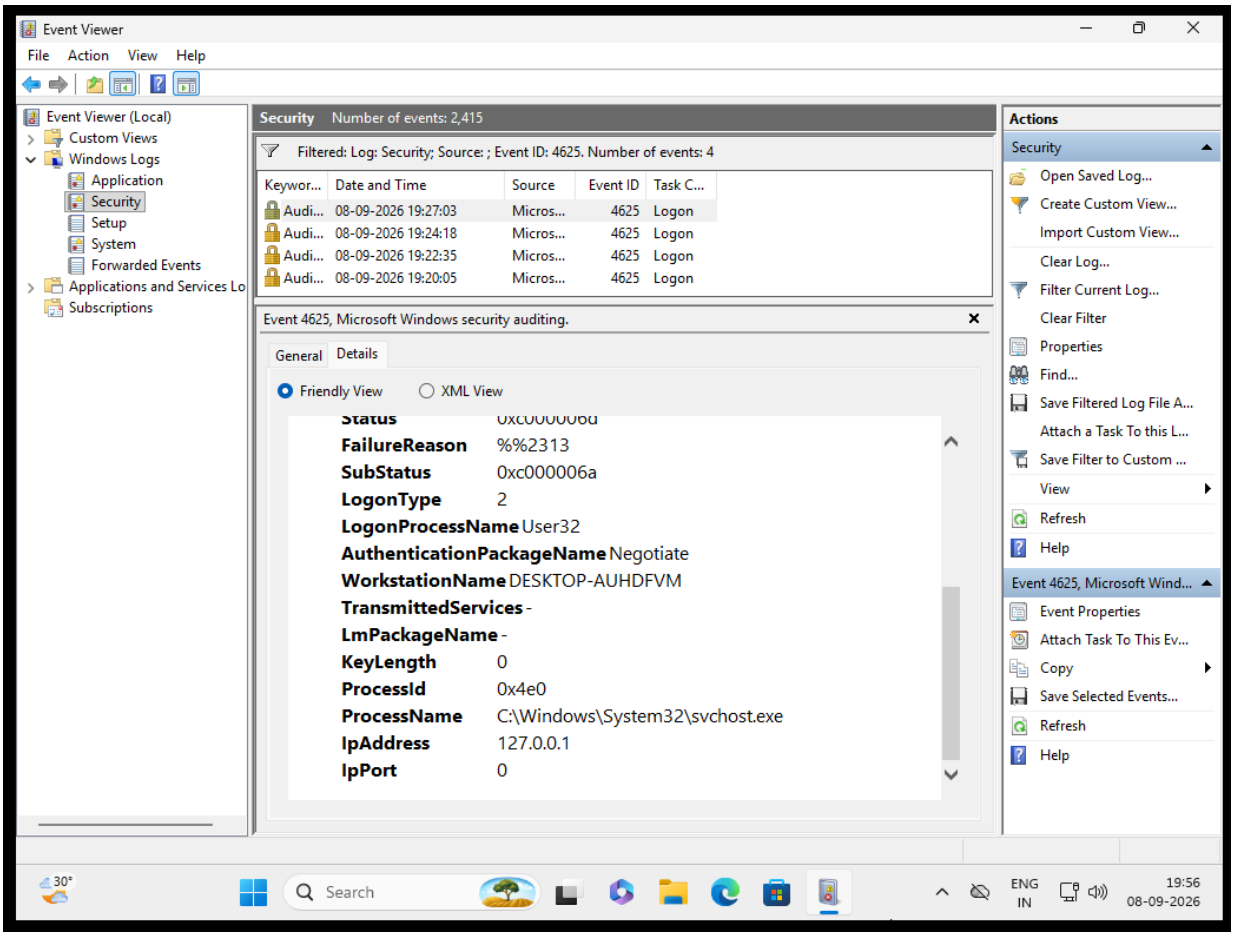
The General tab's readable summary: "An account failed to log on," with the Subject shown as SYSTEM (the process that recorded the event, not the target of the login) and, further down, "Account For Which Logon Failed: CyberAmy," with Logon Type 2.

The Details tab, switched to Friendly View, exposes the individual structured fields behind that summary, which is where the real diagnostic value sits.



The Details tab in Friendly View, showing the individual structured fields Windows recorded for this specific event.

- **TargetUserName: CyberAmy**: confirms exactly whose account was targeted by this attempt.
- **Status: 0xc000006d**: the general failure code, indicating only that the logon did not succeed, without saying why.
- **SubStatus: 0xc000006a**: the specific technical reason for the failure, decoded in detail below.
- **LogonType: 2**: confirms this was an interactive attempt made directly at the machine itself, not over a network.



The remaining fields from the same event, continued further down in the Details view.

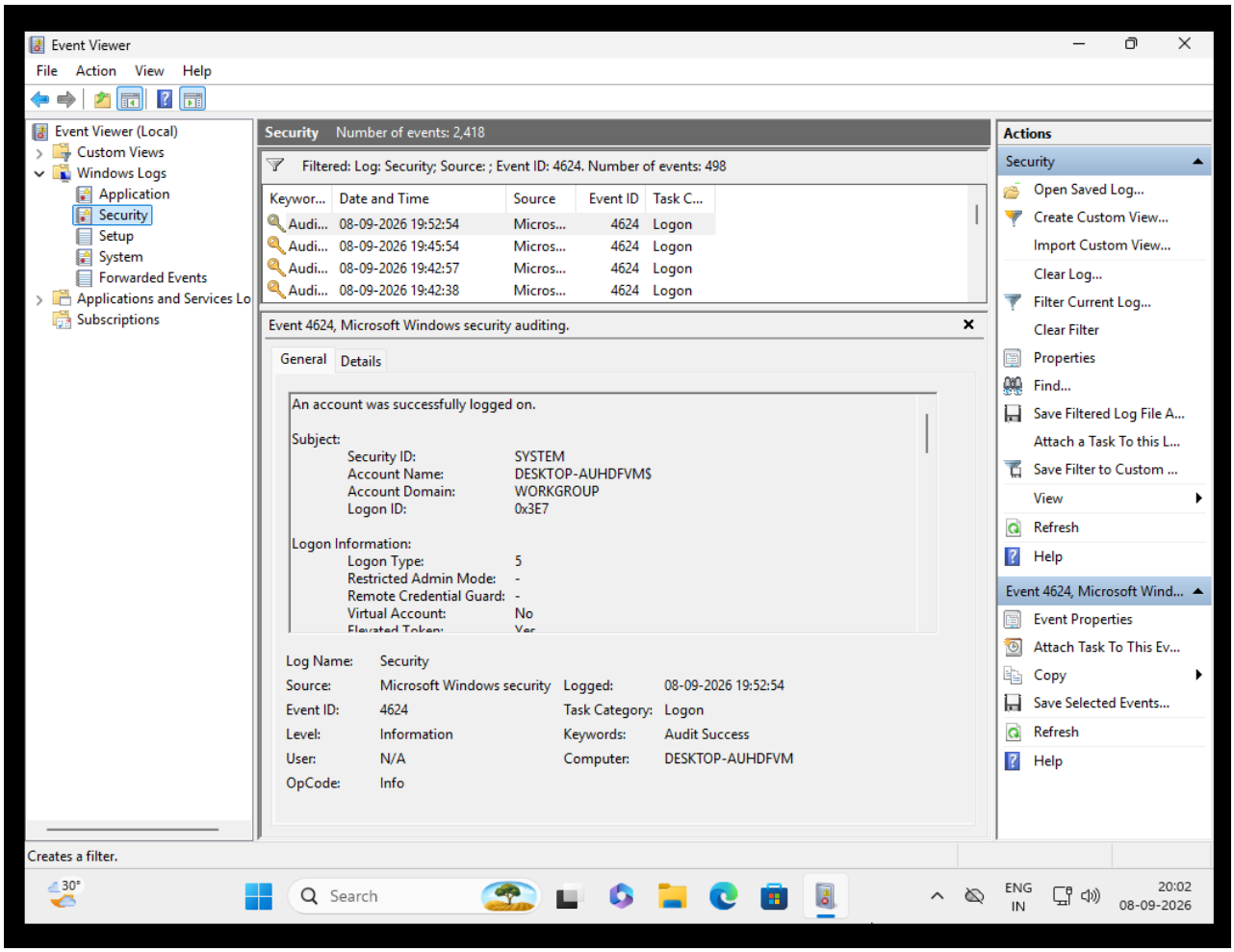
- **AuthenticationPackageName: Negotiate**: identifies the authentication protocol Windows used to process this attempt.
- **IpAddress: 127.0.0.1**: the loopback address, confirming the attempt came from the local machine itself, not across a network.
- **IpPort: 0**: no real network port was involved, consistent with a local, non-network logon.

Two of these fields carry the most weight for an analyst and deserve a full explanation rather than just being read off. The SubStatus code 0xc000006a specifically means STATUS_WRONG_PASSWORD. This is meaningfully different from a code like 0xc0000064 (STATUS_NO_SUCH_USER, meaning the username itself does not exist). The SubStatus seen here confirms that CyberAmy is a real, valid account, and only the password entered was incorrect, exactly consistent with what actually happened. If an investigation ever showed the other code instead, that would point toward someone guessing usernames rather than mistyping a real one's password, a meaningfully different and more concerning pattern.

The second key field is IpAddress: 127.0.0.1. This is the loopback address, the address a machine uses to refer to itself. Seeing this rather than a real network address confirms the login attempt happened directly at the machine's own console, physically or virtually, at the keyboard, not from anywhere across a network. Combined with Logon Type 2 (Interactive), this event is unambiguously a local sign-in attempt, not a remote or network-based one, which matters because remote authentication failures (Logon Type 3, 10, etc.) generally carry a higher baseline concern in a real environment than a local one does.

10. Locating the Corresponding Successful Logon

The filter was changed from 4625 to 4624 to locate the matching successful login. The very first result in this filtered view was not actually the correct one, and that mistake is worth documenting rather than hiding, since it demonstrates a real and useful lesson about reading log data carefully rather than trusting the first result by assumption.



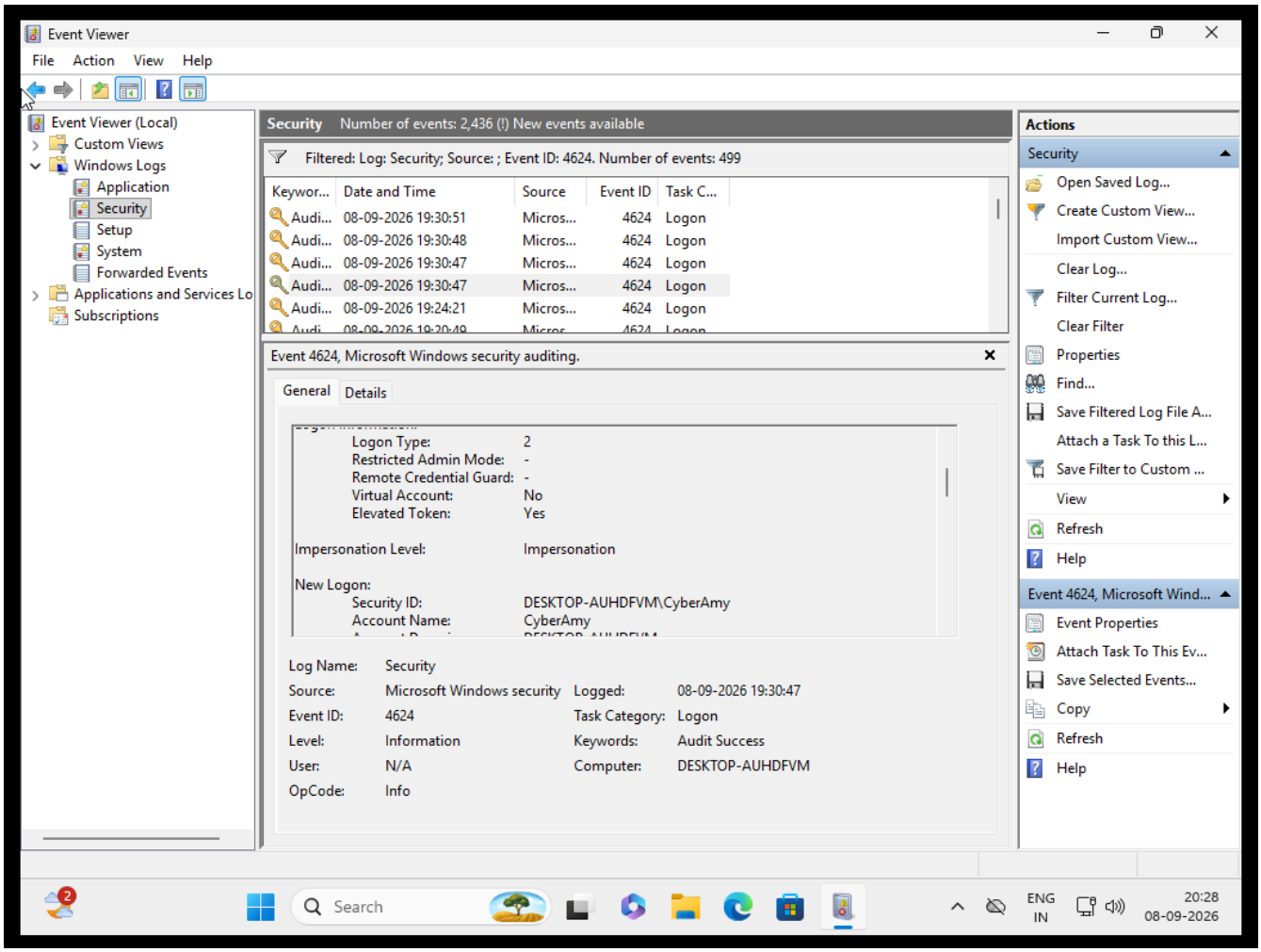
The top result of the 4624 filter, examined field by field before being ruled out.

- **Security ID: SYSTEM**: indicates this event was generated by a core Windows system process, not a person.
- **Account Name: DESKTOP-AUHDFVM$**: the computer's own machine account (the dollar sign is the standard notation for a computer account), not a human user.
- **Logon Type: 5**: Service, confirming this was an automatic background logon, not an interactive one.
- **Process Name: services.exe**: the Windows Service Control Manager, the component responsible for starting background services, further confirming this is routine automated activity.

This is a routine background service logon generated automatically by Windows, unrelated to the actual human login being investigated.

With 499 total Event ID 4624 entries in this log, the overwhelming majority are exactly this kind of routine, automated service activity, not human logins. Assuming the newest or topmost entry is automatically the relevant one is a mistake worth being explicit about, since it is an easy one to make under time pressure during a real investigation. The correct approach, used here, was to check the actual Account Name and Logon Type fields on each candidate event rather than relying on its position in the list.

Working forward from the timestamp of the last failed attempt (19:27:03), the correct successful logon was located a few rows down.



The correct event, confirmed field by field rather than assumed from its position in the list.

- **Timestamp: 19:30:47**: roughly three and a half minutes after the final failed attempt at 19:27:03, precisely matching the real sequence of events.
- **Account Name: CyberAmy**: confirms this event belongs to the actual account being investigated, not a system or service account.
- **Logon Type: 2**: Interactive, matching the same logon type seen across all four failed attempts, the genuine successful interactive logon.

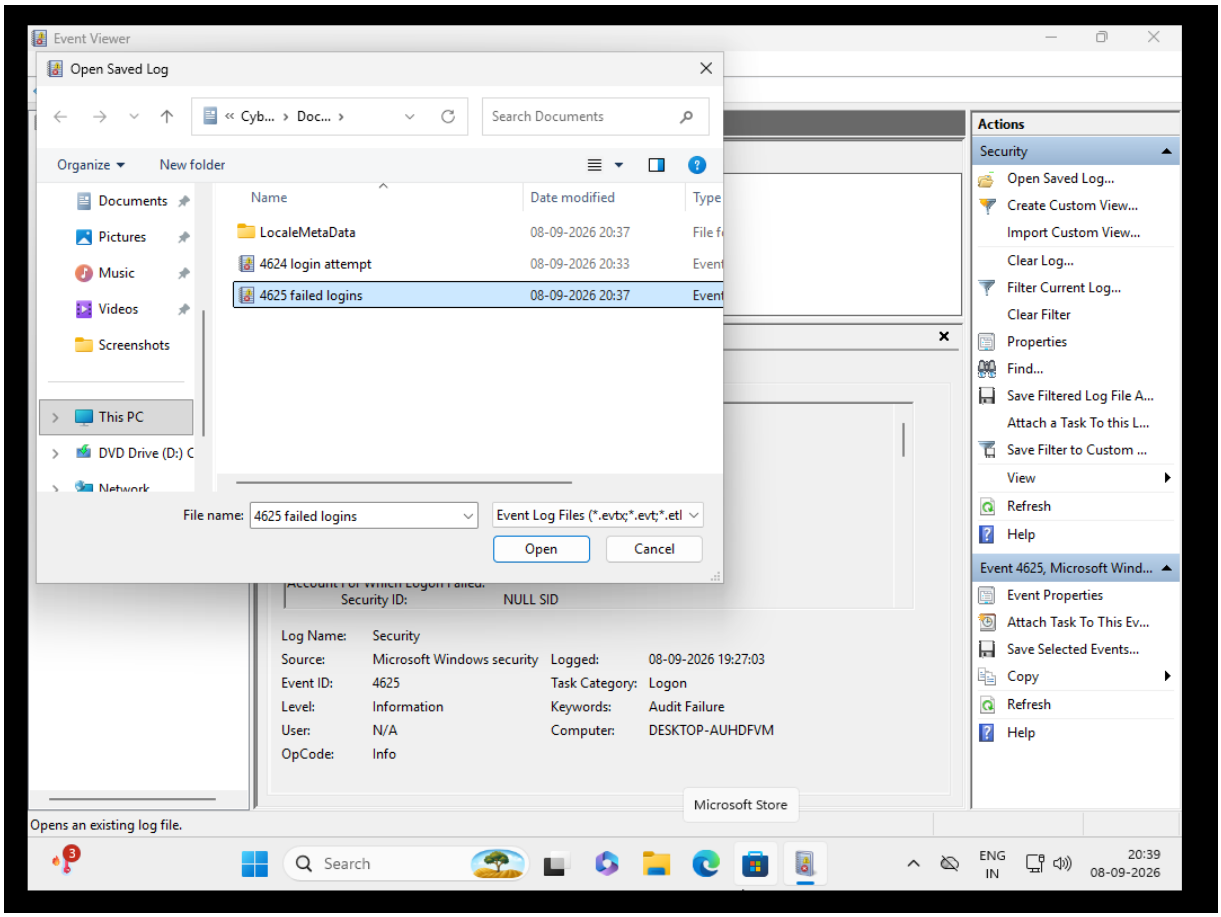
11. Comparing the Failed and Successful Events

Field	Failed attempts (4625)	Successful login (4624)
Count	4	1
Account Name	CyberAmy	CyberAmy
Logon Type	2 (Interactive)	2 (Interactive)
Time range	19:20:05 - 19:27:03	19:30:47
IP Address	127.0.0.1 (local)	127.0.0.1 (local)
Key detail	SubStatus 0xc000006a (wrong password)	Audit Success

Read together, this table tells a coherent, low-concern story: the same account, using the same logon method, from the same local machine, failed a handful of times in quick succession and then succeeded a few minutes later. Every field lines up with an ordinary case of a person struggling to remember their own password, not with any pattern an analyst would associate with an attack, such as attempts against multiple different account names, logons from an unfamiliar network address, or a pattern that continues well past the point a real user would have given up.

12. Preserving the Evidence

Both filtered event sets were exported using "Save Filtered Log File As..." from the Actions panel, saved as .evtx files, the native Windows event log format. This preserves the actual underlying event data, not just a visual snapshot of it, meaning either file could be reopened in Event Viewer later, re-filtered, or handed to someone else for independent review, exactly like the saved text output from the Day 5 baseline task served the same evidentiary purpose on Linux.



Both evidence files saved: "4625 failed logins" and "4624 login attempt," visible in the save dialog alongside their file type (Event Log Files).

13. Findings

13.1 The SubStatus code is more informative than the Status code alone

The general Status code (0xc000006d) only communicates that a logon failed. The SubStatus code (0xc000006a) is what actually tells the analytical story, confirming specifically that the account exists and the password was wrong, rather than the account not existing at all. Any review of 4625 events that stops at the Status field alone is missing the detail that actually matters for judging severity.

13.2 Matching Logon Type across failed and successful events strengthens the benign explanation

Both the failed attempts and the successful login carried Logon Type 2 and originated from 127.0.0.1. A pattern where failed attempts show one logon type or source and a later successful login shows a different one would be far more suspicious, since it could indicate an attacker using a different access path than the legitimate user normally does. Here, every event across the whole sequence used the identical local, interactive method, which is strong supporting evidence for an ordinary, explainable event rather than anything requiring escalation.

13.3 The first result in a filtered log is not automatically the relevant one

Locating the correct 4624 event required rejecting the very first result returned by the filter, which turned out to be an unrelated automated service logon. This is a genuinely useful, transferable lesson: a filter narrows a log down by event type, but it does not know which specific event within that type an analyst actually cares about. Verifying the Account Name and Logon Type fields directly, rather than trusting position or recency, is what actually confirms a match.

13.4 Staying well under the lockout threshold left a wide safety margin

Four failed attempts against a default threshold of 10 left more than enough room for a genuine mistake, such as the accidental fourth attempt while recalling the real password, without any real risk of locking the account. This reflects a sound general practice for safely testing authentication behavior on any live system: know the actual threshold before generating test failures, and stay well clear of it rather than cutting it close.

14. Conclusion

Four safe, intentional failed authentication attempts were generated against a local Windows account, followed by one successful login proving the account was never locked. Both event types were located in the Security log's 2,401 total entries by filtering on their specific Event IDs (4625 and 4624), and the resulting events were read at the field level rather than just acknowledged as present, extracting the account name, logon type, network origin, and the specific SubStatus code that distinguishes a wrong password from a nonexistent account.

Beyond simply generating and finding these events, this exercise surfaced a genuinely useful investigative lesson: the first matching result in a filtered log is not automatically the correct one, and confirming specific field values rather than trusting position or recency is what separates a careful read from an assumption. Both event sets were preserved as .evtx files, ensuring this evidence remains usable beyond this report alone.

Together, these results describe a coherent, low-concern authentication event: one account, one method, one local machine, a short run of genuine password mistakes followed promptly by a correct login, exactly the kind of pattern a SOC analyst should be able to read quickly and confidently rule out as routine, using the same field-level evidence this task was built to practice.